СОДЕРЖАНИЕ

Диаграмма классов (class diagram) 2
Видимость 3
Основные элементы диаграммы классов 4
Уточнение диаграммы классов 6
Виды классов 10
Практическая работа № 9 11

Диаграмма классов (class diagram)

Диаграмма классов в языке моделирования UML относится к структурному типу диаграмм, используется для визуализации структуры классов в системе, атрибутов, методов, интерфейсов и отношений между ними. Применяется при проектировании архитектуры, документировании системы, уточнении требований, а также для поддержки системы.

Диаграмма классов представляет описание структуры классов в системе и их взаимосвязи. Она отображает как статические аспекты системы, включая классы, атрибуты и методы, а также динамические аспекты, такие как связи между объектами и выполнение методов во время выполнения программы.

Диаграмма классов соответствует принципам объектно-ориентированного программирования (ООП) и является одним из базовых инструментов проектирования ООП-систем.

Диаграмма классов соответствует принципу **инкапсуляции**. Он заключается в том, что каждый класс должен скрывать свою реализацию от других классов и предоставлять интерфейс для взаимодействия с ним. Это достигается через использование публичных и приватных методов и атрибутов класса, которые определены на диаграмме.

Диаграмма классов также соответствует принципу **наследования**, который позволяет создавать новые классы на основе уже существующих. Наследование позволяет упростить процесс проектирования системы, так как можно использовать уже реализованные классы и добавлять к ним новые функциональности.

Другим важным принципом, которому соответствует диаграмма классов, является **полиморфизм**. Он позволяет одному методу работать с различными объектами, реализующими один и тот же интерфейс. Диаграмма классов позволяет определить интерфейсы и абстрактные классы, которые могут быть реализованы различными классами.

Видимость

Видимость (visibility) в языке моделирования UML определяет уровень доступности элементов модели. Она определяет, какие части программы могут обращаться к конкретному элементу модели.

В языке UML используется четыре уровня видимости:

1. **Public (публичный «+»)** — элементы с такой видимостью доступны из любой точки модели и внешних модулей. Они могут быть использованы как внутри класса, так и снаружи.
2. **Protected (защищенный «#»)** — элементы с такой видимостью не могут быть использованы вне класса, кроме классов-наследников.
3. **Private (приватный «-»)** — элементы с такой видимостью доступны только внутри класса, в котором они объявлены. Они не доступны из других классов и объектов.
4. **Package (пакетный «~»)** — элементы с такой видимостью доступны всем классам, находящимся внутри одного пакета.

Примечание*: указание видимости осуществляется при объявлении элемента модели с помощью знаков +, #, — и ~, соответственно, для публичного, защищенного, приватного и пакетного типа. Например, при объявлении атрибута с публичной видимостью, он будет помечен знаком «+».

Основные элементы диаграммы классов

Класс – основной элемент моделирования в данной диаграмме, отвечающий за абстракцию реального объекта или сущности системы. Класс включает в себя свойства (**атрибуты**) и поведение объектов (**методы**), которые выступают, как экземпляры класса. На диаграмме класс представляется как прямоугольник, разделенный на 3 части: название, атрибуты и методы.

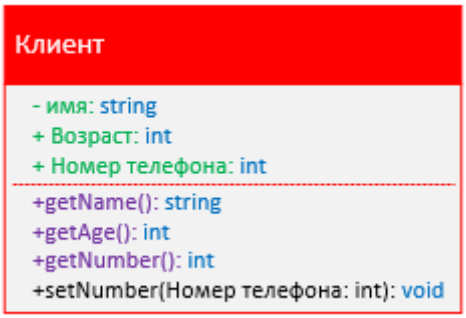


Рисунок 1 – Пример использования класса

Примечание*: **зеленым** цветом отмечены названия атрибутов класса, **фиолетовым** – методы класса, **синим** – их тип данных, а **черным** – методы, которые меняют типы данных.

Исходя из этой структуры мы видим, что название, атрибут и метод класса расположены сверху вниз соответственно.

Зависимость – это отношение между двумя классами, где изменения одного класса могут повлиять на другой. Она обозначается стрелкой, направленной **от зависимого к главному** (или же **от частного к общему**).

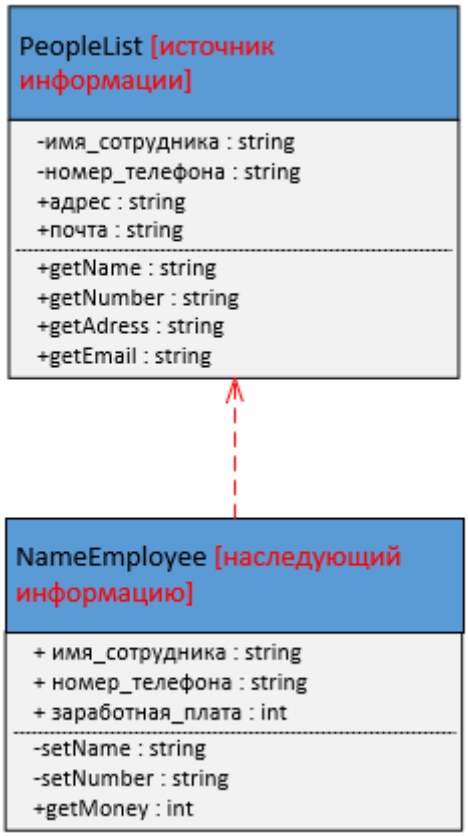


Рисунок 2 – Пример использования зависимости

Уточнение диаграммы классов

Отношения

Агрегация — это отношение между объектами классов, когда один объект является частью другого объекта. В этом случае, объект, содержащий другой объект, называется **контейнером**, а содержащийся объект — **содержимым**. Контейнер может содержать **несколько** объектов-содержимых, а объект-содержимое может быть частью нескольких контейнеров.

В диаграмме классов агрегация обозначается отношением «**больше-чем**» с пустым ромбом на стороне контейнера. Например, если у нас есть класс «Компания» и класс «Сотрудник», то мы можем использовать агрегацию, чтобы показать, что каждая компания содержит несколько сотрудников. В этом случае, класс «Компания» будет контейнером, а класс «Сотрудник» — содержимым.

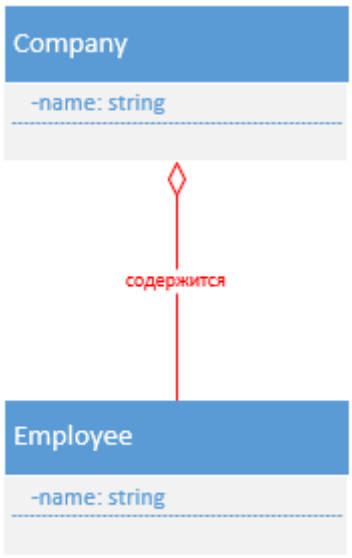


Рисунок 3 – Пример использования агрегации

Примечание*: обратите внимание на пустой ромб на стороне компании, он обозначает что данный класс является контейнером.

Композиция — это более строгий вариант агрегации, когда объект класса-контейнера создает объект класса-части и полностью управляет его жизненным циклом. Объекты класса-части могут принадлежать только **одному** объекту класса-контейнера. При удалении объекта класса-контейнера все объекты класса-части также удаляются.

Композиция обозначается на диаграмме классов UML с помощью пустого ромба на стороне класса-контейнера, соединенной линией с классом-частью. Также можно указать имя роли, которую выполняет класс-часть в отношении с классом-контейнером.

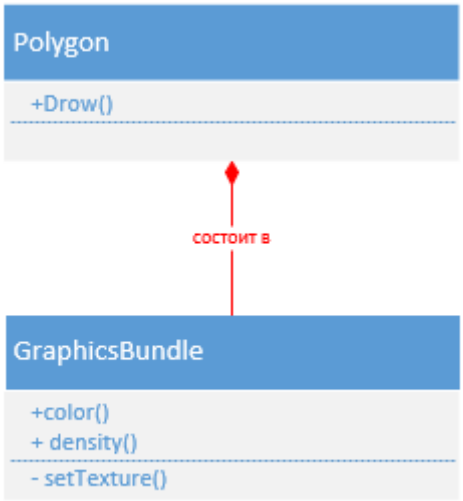


Рисунок 4 – Пример использования композиции

Примечание*: чтобы было проще запомнить разницу между агрегацией и композицией – представьте себе «абьюзивные отношения». Свободный класс – белый (ромб), он может «общаться» (принадлежать) с другими классами. Однако, композиция – это черные отношения (ромб на конце - черный). Частный класс может общаться исключительно с одним – главным классом.

Обобщение — если вспомнить предыдущие лекции, то это уже знакомый для нас вид связи. Это отношение на диаграмме классов UML, которое представляет собой связь между двумя классами, где один класс является более общим (родительским), а другой класс является более конкретным (дочерним). Описывает иерархию классов и позволяет наследовать атрибуты и методы от родительского класса к дочерним классам. Обобщение также называют **наследованием**.

На диаграмме классов обобщение отображается в виде стрелки, которая указывает от дочернего класса к родительскому. Родительский класс обычно находится вверху диаграммы, а дочерние классы располагаются под ним.

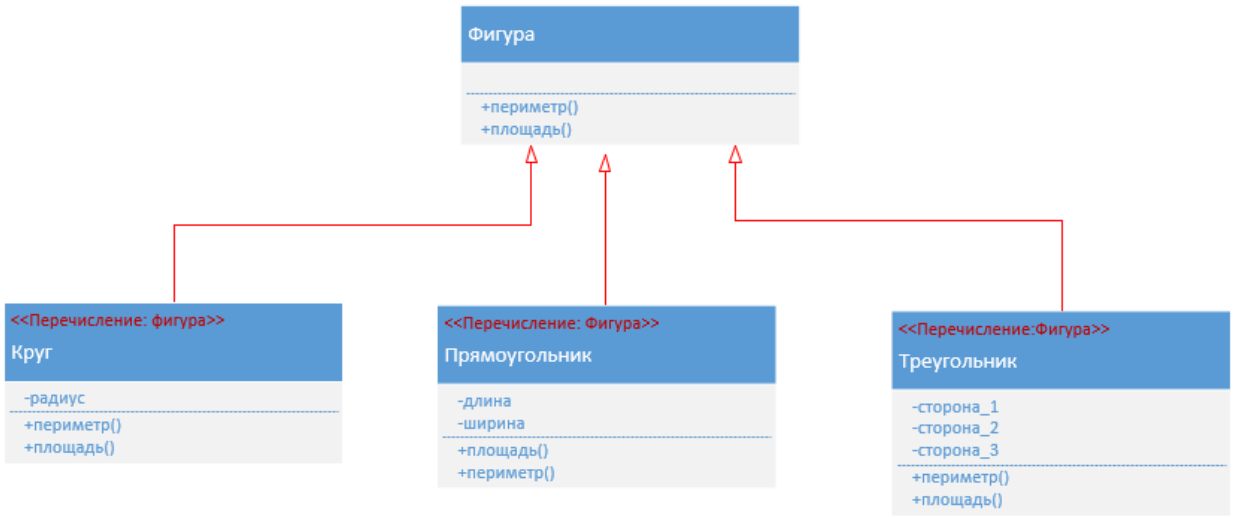


Рисунок 5 – Пример использования наследования

Примечание*: обратите внимание на дочерние классы: в их заголовке обязательно должно быть **перечисление**. Это перечисление помогает понять, к какой группе относятся дочерние классы.

Еще один вид связи – **реализация**. Реализация показывает, что класс реализует интерфейс или абстрактный класс. Класс, который реализует интерфейс, обязан предоставить реализацию всех методов, объявленных в этом интерфейсе. Класс, который наследует абстрактный класс, также должен реализовать все его абстрактные методы.

На диаграмме классов связь реализации обозначается пунктирной линией с треугольной стрелкой, которая указывает на интерфейс или абстрактный класс.

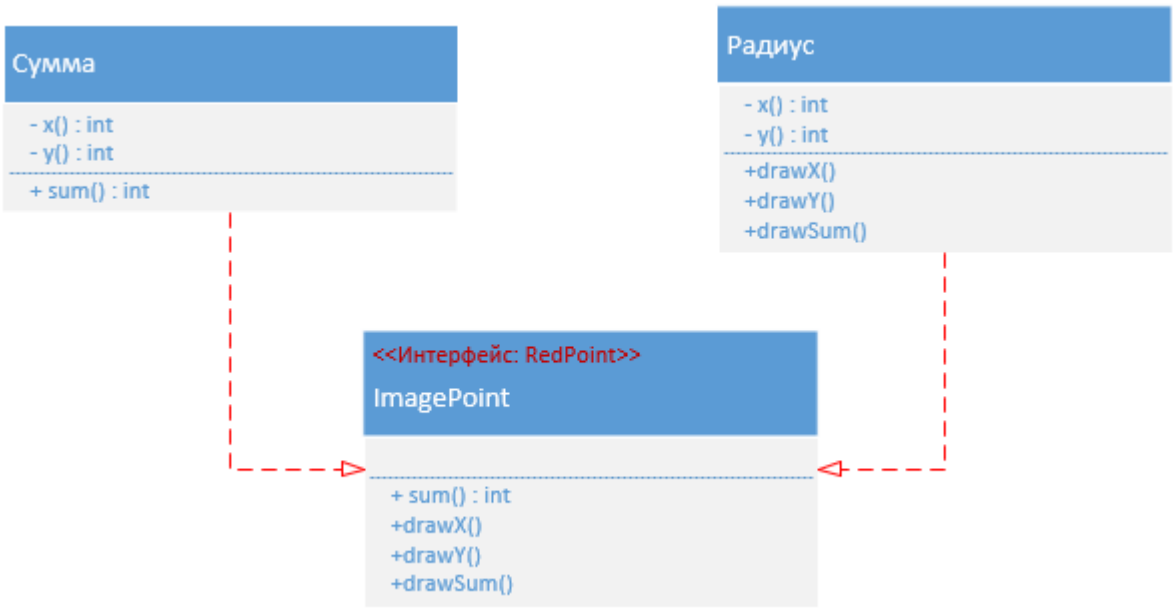


Рисунок 6 – Пример использования реализации

Примечание*: классы «сумма» и «радиус» должны реализоваться в интерфейсе «RedPoint» посредством класса «ImagePoint»

Виды классов

Обычный класс — это класс, который представляет собой абстракцию реального объекта или сущности в системе. Он имеет свои атрибуты и методы, которые описывают характеристики и поведение объектов данного класса.

Абстрактный класс — это класс, который содержит хотя бы один абстрактный метод. Абстрактный метод не имеет реализации в самом классе, а определяется только его сигнатура (имя, параметры, тип возвращаемого значения). Абстрактные классы могут использоваться для определения общих характеристик для нескольких классов, от которых наследуются другие классы.

Интерфейс — это специальный вид класса, который определяет набор методов без их реализации. Интерфейсы используются для определения общих методов, которые должны быть реализованы в классах, которые их имплементируют. Один класс может имплементировать несколько интерфейсов.

Класс перечисления — это класс, который содержит константы, описывающие возможные значения для определенного атрибута. Классы перечисления используются, когда требуется ограничить множество допустимых значений атрибутов.

Класс шаблона — это класс, который представляет общий шаблон для создания конкретных классов, которые определяются позже. Классы шаблоны используются для обобщения классов и уменьшения повторяемости кода.

Каждый класс в диаграмме классов может иметь свою специфическую роль в системе и использоваться для различных целей. Однако, независимо от типа класса, его свойства и методы определяются с целью описания реальных объектов и сущностей в системе.

Практическая работа № 9

Задание 1 (по вариантам)

Важно*: обеспечить работу через классы и предусмотреть интерфейс.

1. Построение графика функции $y = 4x * \frac{1}{x+2}$
2. Построение графика функции $y = 0.25x + x^2$
3. Построение графика функции $y = \frac{1+3.5x}{x^2}$
4. Построение графика функции $y = \tan x + x^{-1}$
5. Построение графика функции $y = \sum 5 + x$
6. Построение графика функции $y = \prod 2 * x$
7. Построение графика функции $y = \sin 2x * \sqrt[3]{x}$
8. Построение графика функции $y = \lim_{x=>0} 64x$
9. Построение графика функции $y = x * 1 - 25x$
10. Построение графика функции $y = 25 * \sqrt[2]{-x}$
11. Построение графика функции $y = e^{34+x} * 2x$
12. Построение графика функции $y = \sin x + \cos -x$
13. Построение графика функции $y = \sin x + \sum 1\frac{2}{x}$
14. Построение графика функции $y = x^2 + 56x$
15. Построение графика функции $y = \frac{e^{-x}}{2x}$
16. Построение графика функции $y = \begin{pmatrix} x & -x \\ 2x & 0 \end{pmatrix}$

Задание 2 (общее)

Важно*: проще всего это задание будет выполнить, исходя из данных какой-либо организации.

Создать диаграмму, включающую в себя реализацию, обобщение, агрегацию и/или композицию. Дополнять предыдущую диаграмму запрещено.